

Laravel File & Folder Structure

Laravel has a very well-organized file/folder structure. Let's take a look at the most important folders and files in a Laravel project.

composer.json

This file contains all the Composer dependencies for the Laravel application. This includes Laravel itself and any other third-party packages you install. For regular dependencies, we have PHP, Laravel and Laravel Tinker. Tinker is a REPL (Read-Eval-Print Loop) for Laravel. It allows you to interact with your Laravel application from the command line.

Let's look at the dependencies in the `composer.json` file:

Dependencies

- `php` - The PHP version required for the Laravel application.
- `laravel/framework` - The Laravel framework itself.
- `laravel/tinker` - The Laravel Tinker package. This is a REPL (Read-Eval-Print Loop) for Laravel. It allows you to interact with your Laravel application from the command line.

Development Dependencies

- `phpunit/phpunit` - The PHP Unit testing framework.
- `fakerphp/faker` - A PHP library that generates fake data.
- `laravel/pint` - A browser automation and testing tool for Laravel.
- `laravel/sail` - A light-weight command-line interface for interacting with Laravel's default Docker development environment.
- `mockery/mockery` - A simple yet flexible PHP mock object framework for use in unit testing with PHPUnit or PHPSpec.
- `nunomaduro/collision` - An error reporting tool for command-line applications

PSR Autoload Mapping

Let's look at the namespaces being autoloaded using PSR (PHP Standard Recommendation). The PSR-4 standard maps namespaces to directory paths, making it easier to autoload classes without needing to manually include files.

In the Laravel `composer.json` file, you will see a section called `autoload`. This section is used to specify the autoloading rules for the Laravel application.

It looks like this:

```
"autoload": {  
    "psr-4": {  
        "App\\": "app/",  
        "Database\\Factories\\": "database/factories/",  
    }  
}
```

```
        "Database\\Seeders\\": "database/seaders/"
    },
}
```

What this is doing is mapping the `App` namespace to the `app/` directory, the `Database\\Factories` namespace to the `database/factories/` directory, and the `Database\\Seeders` namespace to the `database/seaders/` directory. This makes it easier to autoload classes without needing to manually include files.

So if we wanted to use a class in the `App` namespace, we would simply use the `App` namespace and the class name. Composer would then autoload the class from the `app/` directory.

package.json

This file contains all the NPM dependencies for the Laravel application. This may seem strange since Laravel is a PHP framework, but often times, Laravel is used as a backend for a JavaScript frontend. This file is used to manage the JavaScript dependencies for the frontend. It includes dependencies like Axios, which is an HTTP client as well as Vite, which is a build tool and frontend development server.

.env

This file contains all the environment variables for the Laravel application. This is where you configure things like the database connection, mail settings, etc. This file is not committed to version control since it contains sensitive information. There is also a `.env.example` file which contains the default values for the environment variables. This one is committed to version control.

vite.config.js

This file contains the configuration for Vite, which is a build tool and frontend development server. This file is used to configure things like the build target, the base URL, etc.

You will also see a file called `artisan`. This is the command-line interface for Laravel. You can use this to run commands like `php artisan serve` to start the development server.

.editorconfig

This file contains the configuration for the editor. This is used to maintain consistent coding styles across different editors and IDEs.

.gitignore

This file contains all the files and folders that should be ignored by Git. This is where you specify the files and folders that should not be committed to version control.

.gitattributes

This file contains the attributes for the files and folders in the Git repository. This is used to specify things like the line endings, the merge strategy, etc.

Let's look at some of the important folders:

- **app** - Contains all the models, controllers, and other PHP classes. Laravel is a Model-View-Controller (MVC) framework. This is where the models and controllers live. Models are used to interact with the database and controllers are used to handle requests and responses. The controllers are in the **app/Http** folder.
- **bootstrap** - Contains the files that bootstrap the Laravel application. This is where the application is initialized.
- **config** - Contains all the configuration files for the Laravel application. You can configure things like the database connection, mail settings, etc.
- **database** - Contains all the database related files. This is where the migrations and seeders live. You also get a default SQLite database file. By default Laravel uses SQLite as the database, but you can easily change it to MySQL, PostgreSQL, or any other database.
- **public** - Contains the public files for the Laravel application. This is where the CSS, JS, and image files live. This is also where the **index.php** file lives which is the entry point for the application.
- **resources** - Contains all the views, language files, and other resources for the Laravel application. This is where the Blade templates live.
- **routes** - Contains all the route files for the Laravel application. This is where you define the routes for your application. This is one of the first places you will look when you are trying to understand how the application works.
- **storage** - Contains all the storage files for the Laravel application. This is where the logs, cache, and other storage files live.
- **tests** - Contains all the test files for the Laravel application. This is where you write your tests.
- **vendor** - Contains all the Composer dependencies for the Laravel application. This is where all the third-party packages live.